

Lua Cutscenes without Lua Experience

没有 Lua 经验的 Lua 过场动画



Gamation

Follow

36 min read · Jul 30, 2023



4

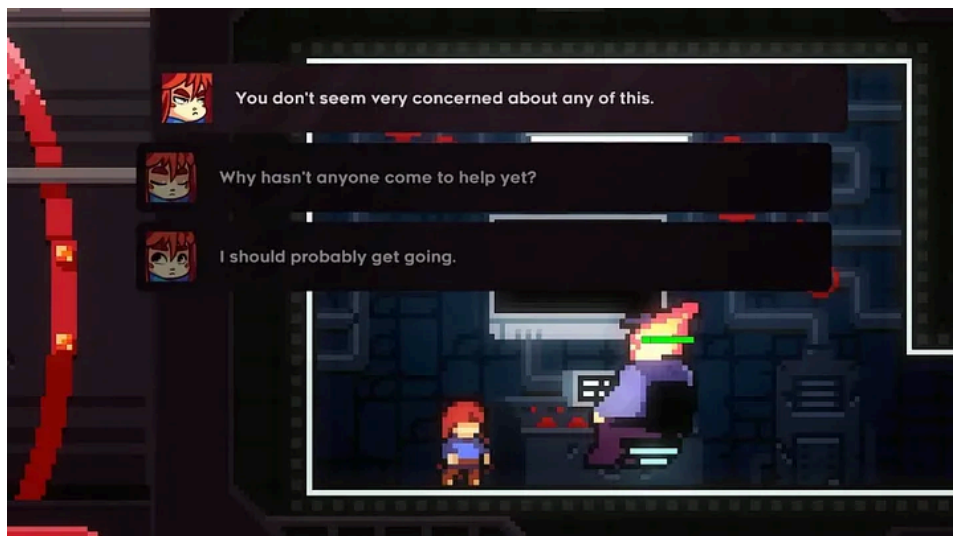


2



So you're probably like me and have no idea how code works, but you want to make those cool cutscenes like you've seen in maps like paint or Perceptions. Congrats for being a part of the intended audience!

所以你可能和我一样，对代码一窍不通，但你想制作那些酷炫的过场动画，就像你在 paint 或 Perceptions 等地图里看到的那样。恭喜你成为目标受众！



A lua cutscene in action. Credit: Soloiini

正在运行的 Lua 过场动画。图源：Soloiini

As I worked on Strawberry Jam, I gained a lot of general Lua Cutscenes experience, mostly through pain and suffering. Everyone who's looked at the official documentation comes away with the opinion that it's severely lacking, and while some attempts have been made to construct better documentation ([like microlith's here](#), which is actually quite good) I have yet to see a full "beginner's guide" showing how to do many common things like walking, dialogue, dashing, or even just turning around. So here we are — I

suck at writing introductions, so let's just start.

在制作 **Strawberry Jam** 的过程中，我积累了大量的 **Lua** 过场动画经验，主要还是通过痛苦和挣扎。所有看过官方文档的人都会觉得它严重不足，虽然已经有人尝试构建更好的文档（比如这里的 **microlith**，实际上相当不错），但我还没有看到一本完整的“入门指南”，展示如何实现走路、对话、冲刺，甚至只是转身等许多常见功能。所以现在我们来到了这里——我写引言很烂，那就直接开始吧。

Chapter 0: Mod Setup 第 0 章：模组设置

Before you can start coding cutscenes, you have to know how to put them in-game. This is going to be a fairly short section going over how to get a cutscene running — we'll assume for now that this cutscene, which we're going to call `myCutscene.lua`, has perfectly functional code inside it.

在开始编写过场动画之前，你必须先知道如何将它们放入游戏中。这一节将简要介绍如何让过场动画运行起来——我们暂时假设这个过场动画（我们将其称为 `myCutscene.lua`）内部拥有完全正常的代码。

Start by creating a folder in your mod root (`Celeste/Mods/YourModFolder/`) titled **Assets**. Then create subfolders for your mod name and your username, just so Everest doesn't get confused. When you're all done, you'll have a filepath that looks like

```
Celeste/Mods/YourModFolder/Assets/YourUsername/YourModName/ .
```

首先，在你的模组根目录（`Celeste/Mods/YourModFolder/`）中创建一个名为 **Assets** 的文件夹。然后为你的模组名称和用户名创建子文件夹，以免 **Everest** 混淆。完成后，你将得到一个类似

```
Celeste/Mods/YourModFolder/Assets/YourUsername/YourModName/ 的文件路径。
```

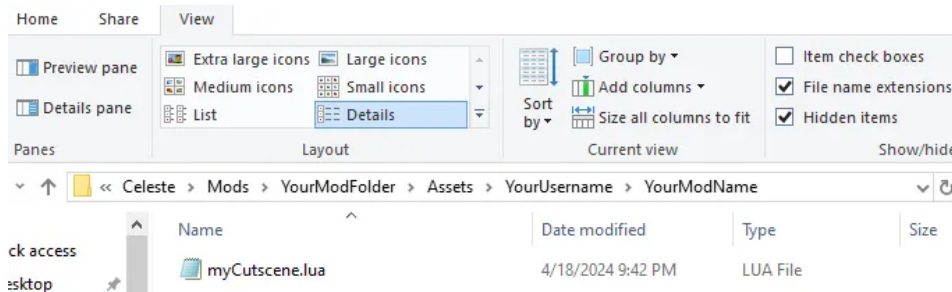
Inside the `YourModName` folder, create a new text file. On Windows, you do this by right clicking anywhere inside the folder, then selecting **New**, and then text file from the menu that pops up. Name this file whatever you want, though like I said, we're going to call it `myCutscene` .

在 `YourModName` 文件夹内，创建一个新的文本文件。在 **Windows** 系统中，你可以通过在文件夹内任意位置右键点击，然后选择“新建”，再从弹出的菜单中选择“文本文档”来完成这一操作。你可以将这个文件命名为你想要的任何名称，不过，正如我所说，我们将将其命名为 `myCutscene` 。

Now make sure you have file extensions turned on — on Windows, you can enable this by clicking on the “View” menu on top of the File Explorer window and then checking the box that says “show file extensions.” Go to rename your text file once again (right click it and select **Rename**, or press **F2** while it's selected). Change the file extension from `.txt` to `.lua`. Your computer may warn you about this with some text about the file “becoming

unstable” that you can safely ignore.

现在确保文件扩展名已开启——在 Windows 系统中，你可以在文件资源管理器窗口顶部的“查看”菜单中点击，然后勾选标有“显示文件扩展名”的复选框。再次重命名你的文本文件（右键点击它并选择“重命名”，或选中它时按 F2 键）。将文件扩展名从 .txt 改为 .lua 。你的电脑可能会显示一条警告信息，说文件“变得不稳定”，你可以安全地忽略这条信息。



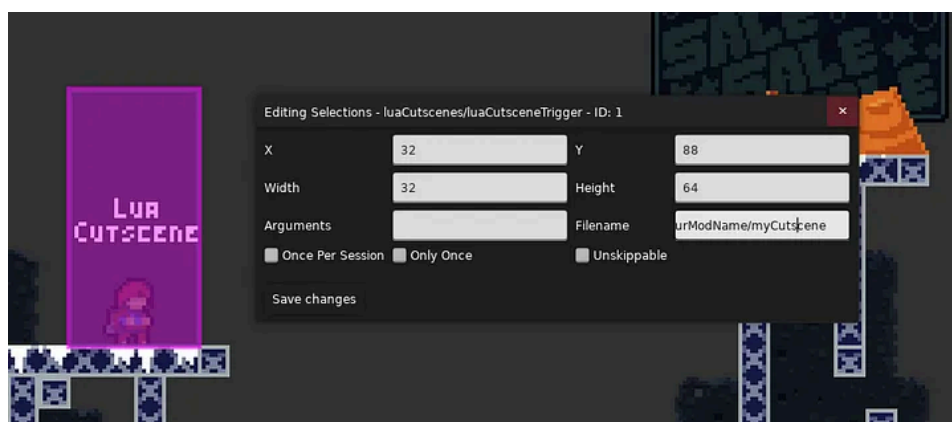
It'll look like this. Remember to rename the folders to fit with your name and mod name!

它看起来会像这样。记得将文件夹重命名，以匹配你的用户名和模组名！

Now, open your map editor of choice and place down a Lua Cutscene trigger from the Lua Cutscenes mod (or a Lua Talker entity, but this tutorial isn't about those.) Right click the trigger and in the `Filename` field add the full path to your cutscene file starting from `Assets`. For us, this would be `Assets/YourUsername/YourModName/myCutscene` . Do not include the file extension or anything before the `Assets` folder.

现在，打开你选择的地图编辑器，并从 Lua Cutscenes 模组中放置一个 Lua Cutscene 触发器（或者是一个 Lua Talker 实体，但本教程不涉及这些）。右键点击触发器，在 `Filename` 字段中添加从 `Assets` 开始的完整路径到你的 cutscene 文件。对我们来说，这将是

`Assets/YourUsername/YourModName/myCutscene` 。不要包含文件扩展名或 `Assets` 文件夹之前的任何内容。



You can't see it, but it reads `Assets/YourUsername/YourModName/myCutscene` — trust me.

虽然你看不到，但它读取的是 `Assets/YourUsername/YourModName/myCutscene`——相信我。

Congratulations! You've made a working cutscene. If nothing is happening when you enter the trigger, you may need to restart the game if you added

the cutscene recently. And speaking of cutscenes — let's get to coding them!

恭喜！你已经制作了一个可用的过场动画。如果你最近添加了过场动画，进入触发器时如果什么都没发生，你可能需要重新启动游戏。说到过场动画——让我们开始编写它们吧！

Section TL;DR: TL;DR 部分:

To get a cutscene working, create a `.lua` file in

`Assets/YourUsername/YourModName/` (and sub out those placeholders for the appropriate names.) Put the path to that file in a lua cutscenes trigger and bam, you're set.

要让过场动画正常工作，在 `Assets/YourUsername/YourModName/` 中创建一个 `.lua` 文件（并将这些占位符替换为适当的名称）。将这个文件的路径放入一个 lua 过场动画触发器中，搞定！

Chapter 1: The Basics 第1章：基础知识

This is the base function in Lua Cutscenes. Everything you put inside `onBegin()` will run as soon as the player enters the cutscene trigger.

这是 Lua Cutscenes 中的基础函数。你放入 `onBegin()` 中的所有内容，将在玩家进入过场动画触发器时立即运行。

```
function onBegin()  
    say("DIALOGUE_ID")  
end
```

(If instead of a cutscene trigger, you're using a Lua Talker NPC, you'll want to use the related function `onTalk()` instead. However, for the rest of this tutorial, we're going to use `onBegin()` as if you were using a typical trigger.)

(如果你们不是使用场景触发器，而是使用 Lua Talker NPC，你们需要使用相关的 `onTalk()` 函数。然而，在本教程的其余部分，我们将像使用典型触发器一样使用 `onBegin()` 。)

```
function onTalk()  
    say("DIALOGUE_ID")  
end
```

You may have heard in some programming languages that a semicolon is required to end a line of code. Thankfully, this is optional in Lua. Lua runs through all lines in the function sequentially, and knows when a function is over by the word `end`. In these examples above, I used the `say()` function

without explicitly explaining it. So let's get into that!

你们可能在某些编程语言中听说过，代码行需要用分号结束。幸运的是，在 Lua 中这是可选的。Lua 会按顺序运行函数中的所有行，并通过 `end` 这个词知道函数何时结束。在上述示例中，我使用了 `say()` 函数而没有明确解释。那么让我们进入正题吧！

The `say()` function reads a dialogue ID taken from your `English.txt` file. (If you need help establishing an `English.txt`, there's a good article on it on the Everest wiki [here](#).) Simply writing `say("...")` will cause a textbox to pop up in game as if the player had entered a dialogue trigger with the same ID.

`say()` 函数读取从你们的 `English.txt` 文件中获取的对话 ID。（如果你需要关于建立 `English.txt` 的帮助，Everest 维基上有一篇很好的文章。）简单地写 `say("...")` 会在游戏中弹出一个文本框，就像玩家输入了具有相同 ID 的对话触发器一样。

This function provides an opportunity for us to look at some common Lua syntax. One common “data type” in Lua and other programming languages is a string, which is a set of characters arranged in a specific order. That sounds complicated, but really it's just any sentence.

这个函数为我们提供了一个机会，让我们来看看一些常见的 Lua 语法。Lua 和其他编程语言中常见的一种“数据类型”是字符串，它是一组按特定顺序排列的字符。听起来很复杂，但实际上它只是任何句子。

```
local myString = "This is a string!"
```

Strings, like dialogue IDs, need to be surrounded by quotes to be recognized as strings. As already mentioned, you don't need a semicolon to end a line; you also don't need the `helpers` prefix like the official documentation would make you think. Comments in Lua are written using 2 dashes. (A comment is some text that the computer will ignore when running, so it allows you to write messages and documentation.)

字符串，如对话 ID，需要用引号包围才能被识别为字符串。正如之前提到的，你不需要分号来结束一行；你也不需要像官方文档那样使用 `helpers` 前缀。Lua 中的注释是用两个破折号写成的。（注释是一些计算机在运行时会忽略的文本，因此它允许你写消息和文档。）

```
say("DIALOGUE_ID") -- good
say(DIALOGUE_ID) -- bad, will not work
helpers.say("DIALOGUE_ID"); -- will work, but has unnecessary stuff
```

Lua is case sensitive, so,

Lua 区分大小写，所以，

```
say("DIALOGUE_ID") -- works, if you define DIALOGUE_ID
say("dialogue_id") -- does not work if *only* DIALOGUE_ID is defined
Say("DIALOGUE_ID") -- does not work!
SAY("DIALOGUE_ID") -- does not work!
```

It's considered convention to define everything in lowercase. Names cannot contain spaces, so if you need multiple words, you should capitalize the first letter of every word after the first, e.g. `someLongSentence`. Most functions will be case sensitive, too. Everest will consider `DIALOGUE_ID` and `dialogue_id` to be two separate dialog entries. Pick one convention and stick to it.

按惯例，所有内容都应小写定义。名称中不能包含空格，所以如果你需要多个单词，应该在第一个单词之后将每个单词的首字母大写，例如

`someLongSentence`。大多数函数也会区分大小写。Everest 会认为 `DIALOGUE_ID` 和 `dialogue_id` 是两个不同的对话条目。选择一种惯例并坚持使用它。

The next simple function I'd like to introduce is the `wait()` function. Unlike `say()`, which takes a string, `wait()` takes a number measured in seconds. Since it's not a string, you don't need to put quotes around it, and putting no number at all will result in a wait time of 1 frame.

我想介绍的下一个简单函数是 `wait()` 函数。与 `say()` 不同，`say()` 接受字符串，`wait()` 接受以秒为单位的数字。因为它不是字符串，所以你不需要用引号包围它，并且完全不输入数字将导致等待时间为 1 帧。

```
function onBegin()
  wait(1) -- wait 1 second
  say("DIALOGUE_ID_1") -- say the line
  wait(1.5) -- wait 1.5 seconds
  say("DIALOGUE_ID_2") -- say the second line
  wait() -- wait 1 frame
end
```

The third “basic” function (or rather, function family) is the `walk()` function and its related basic movement functions. These allow you to move Madeline left or right. She can't jump or dash, but for basic movement on flat ground, it's worth knowing these.

第三个“基本”函数（或者更准确地说，函数家族）是 `walk()` 函数及其相关的基本移动函数。这些函数允许你将玛德琳向左或向右移动。她不能跳跃或冲刺，但对于在平坦地面上的基本移动，了解这些函数是值得的。

`walk()` is the most basic of the movement functions. It takes a number inside the parenthesis — remember for numbers you don't have to put quotes around them — and moves Madeline that many pixels. Positive numbers have her move right, negative numbers have her move left. She'll

play the walking animation while doing this.

`walk()` 是最基本的移动函数。它接受括号内的一个数字——记住，对于数字你不需要用引号包围它们——并将玛德琳移动那么多像素。正数让她向右移动，负数让她向左移动。她会在移动时播放行走动画。

```
function onBegin()
  say("DIALOGUE_ID_1")
  walk(8) -- move 8 pixels right
  walk(-16) -- move 16 pixels left
end
```

This “number” differs from the number we used with the `wait()` example earlier because it must be an integer. Like strings earlier, an integer or “int” is another data-type used in lua. They can be positive, negative, or zero, but they must be whole numbers.

这个“数字”与我们之前在 `wait()` 示例中使用的数字不同，因为它必须是一个整数。像字符串一样，整数或“int”是 Lua 中使用的另一种数据类型。它们可以是正数、负数或零，但必须是整数。

Note that `walk()` doesn’t only take an integer in between its parentheses. What we put inside the parentheses are called “parameters,” and `walk()` actually takes up to 4 of them: `distance`, `walkBackwards`, `speedMultiplier`, and `keepWalkingIntoWalls`. Functions in lua can take more than one parameter, and `walk()` proves to be a pretty good example of this. To customize the function more, just add what you want the values of each parameter to be in the “function call” (the line with the function name and parentheses in it) and separate them with commas. That’s a bunch of technical language — here’s what it looks like in practice.

请注意，`walk()` 不仅在其括号内接受一个整数。我们放入括号内的被称为“参数”，而 `walk()` 实际上最多可以接受 4 个参数：`distance`、`walkBackwards`、`speedMultiplier` 和 `keepWalkingIntoWalls`。Lua 中的函数可以接受多个参数，`walk()` 就是一个很好的例子。要更详细地自定义函数，只需在“函数调用”（包含函数名和括号的行）中添加每个参数的值，并用逗号分隔。这堆技术术语——以下是其实际应用的样子。

```
function onBegin()
  walk(8) -- move 8 pixels right
  walk(8, true) -- move 8px and do not change facing angle before doing so
  walk(8, true, 2) -- move 8px at 2x speed and do not change facing angle
  walk(8, true, 2, true) -- move 8px and 2x speed, do not change facing angle,
                        -- do not stop walking even if you run into a wall
end
```

All of these functions are valid and will run. I’m not going to go into the parameters of every single function listed here from here on, as it would

take too much time; however, for once the official documentation is somewhat useful. Some are listed [here under the helper functions page](#), but note that this list is incredibly incomplete. If I don't mention a parameter, assume it's used relatively infrequently and doesn't need to be mentioned.

所有这些函数都是有效的，并且可以运行。我不会从现在开始逐一介绍这里列出的每个函数的参数，因为这会花费太多时间；然而，官方文档偶尔还是有些有用的。其中一些被列在辅助函数页面下，但请注意这个列表非常不完整。如果我没有提到某个参数，请假设它使用频率相对较低，不需要特别说明。

As I hinted towards earlier, there are other movement functions besides just `walk()`, although I'd recommend using `walk()` as much as possible because the others are much more complex. I'll go through them now, but in a bit less depth:

正如我之前暗示的，除了 `walk()` 之外还有其他移动函数，但我建议尽可能多地使用 `walk()`，因为其他的要复杂得多。我现在会逐一介绍它们，但会稍微浅一些：

- `run()` is functionally the same as `walk()`, but uses a running animation instead of a basic walk.

`run()` 在功能上与 `walk()` 相同，但使用的是跑步动画而不是基本的行走。

- `walkTo()` is where things start getting complex. Instead of taking a pixel amount from the player's current position, it requires a position in "world coordinates" and the player will walk to there. You can see these coordinates by opening up the debug console — they should be in the top left. If you have Another Lonn Plugin installed, you can also get them by pressing tilde in Lonn. **The specific type of coordinates you're looking for are the ones following just "world."** For more information on this one, [see the official documentation](#).

`walkTo()` 是从这里开始变得复杂的。它不是从玩家的当前位置获取像素量，而是需要“世界坐标”中的位置，玩家会走向那里。你可以通过打开调试控制台看到这些坐标——它们应该在左上角。如果你安装了 Another Lonn 插件，也可以通过按 Lonn 中的波浪号来获取它们。你正在寻找的具体坐标类型是紧随“world”之后的那些。有关此函数的更多信息，请参阅官方文档。

- `runTo()` is the same as above but uses the running animation.

`runTo()` 与上面相同，但使用的是跑步动画。

- `jump()` makes the player jump in place.

`jump()` 使玩家原地跳跃。

The final two functions I'd consider to be basic are very simple and don't require any parameters. These functions, `makeUnskippable()` and `disableMovement()` respectively, do exactly what you'd expect them to. `disableMovement()` is especially useful if Madeline is moving in your cutscene, as it means the player won't be able to break the intended

movement path by just pressing WASD or moving the stick.

我考虑的最后两个基本功能非常简单，不需要任何参数。这两个功能，`makeUnskippable()` 和 `disableMovement()` 分别，做的是你期望它们做的事情。`disableMovement()` 特别有用，如果 **Madeline** 在你的过场动画中移动，这意味着玩家不会通过简单地按 WASD 或移动摇杆来破坏预期的移动路径。

```
function onBegin()  
  makeUnskippable()  
  disableMovement()  
  say("DIALOGUE_ID")  
  run(16)  
  run(-24)  
end
```

Note that the cutscene above will leave the player permanently stuck in place. This is because as soon as the `onBegin()` function ends, so does the cutscene. We can amend this by adding another function, called `onEnd()`, to the file. This function is automatically run after `onBegin()` finishes, and it will be the last thing run in the cutscene. However, `onEnd()` has some quirks that can very easily lead to it crashing your game. This is because, unlike all the functions we’ve gone over before, `onEnd()` is not a “coroutine”. Celeste uses this special kind of function — coroutines — to run a function over time. To define in a more intuitive way what a coroutine is, it’s useful to go back to `say()` for a second.

请注意，上面的过场动画会让玩家永久地卡在原地。这是因为一旦 `onBegin()` 功能结束，过场动画也就结束了。我们可以通过向文件中添加另一个名为 `onEnd()` 的功能来解决这个问题。这个功能在 `onBegin()` 完成后自动运行，并且将是过场动画中最后运行的东西。然而，`onEnd()` 有一些奇怪的特性，很容易导致游戏崩溃。这是因为与之前我们讨论的所有功能不同，`onEnd()` 不是一个“协程”。Celeste 使用这种特殊的函数——协程——来随时间运行一个功能。为了更直观地定义什么是协程，让我们暂时回到 `say()` 看一看。

Ordinarily, when a function is run by other code, that function has to finish completely before anything else is allowed to run, including e.g. any rendering code that is needed to display a text box to the screen. You could have `say()` simply spawn a text box, but now a sequence of `say()` s will spawn multiple text boxes at the same time, rather than waiting for the previous one to be dismissed. A coroutine is allowed to do some work, and then stop to be resumed later.

通常情况下，当其他代码运行一个函数时，该函数必须完全执行完毕后，才允许其他任何代码运行，包括例如显示文本框所需的渲染代码。你可以让 `say()` 直接生成一个文本框，但现在 `say()` 会同时生成多个文本框，而不是等待前一个被关闭。协程允许做一些工作，然后停止，稍后继续执行。

Conveniently, `onBegin()` is a coroutine, so we can create text boxes and then stop, wait for that text box to be dismissed, and then do something

afterwards (including creating more text boxes). This is what `say()` does. But not all functions are coroutines; as established, the `onEnd()` function is not, so you can't use `say()` (or any other function that "takes time") inside of it without a game crash or a more advanced workaround.

幸运的是，`onBegin()` 是一个协程，因此我们可以生成文本框然后停止，等待该文本框被关闭，之后做些其他事情（包括生成更多文本框）。这就是 `say()` 的作用。但并非所有函数都是协程；正如所建立的，`onEnd()` 函数不是协程，所以你不能在它里面使用 `say()`（或任何其他需要时间的函数）而不导致游戏崩溃或需要更高级的解决方案。

If none of this made sense, that's okay for now. All you need to know is that functions that don't occur and conclude instantaneously, such as waiting for some amount of time or putting up a text box, are a special kind of function that cannot be used in `onEnd()`.

如果以上内容都不理解也没关系。你需要知道的是，那些不会立即发生和结束的函数，例如等待一段时间或显示文本框，是一种特殊的函数，不能在 `onEnd()` 中使用。

Returning to our cutscene example earlier, make sure to use the complementary function `enableMovement()` (which is *not* a coroutine, as it happens instantaneously instead of over time) in `onEnd()` to allow Madeline to move again once the cutscene is done.

回到我们之前提到的过场动画示例，确保在 `onEnd()` 中使用辅助函数 `enableMovement()`（它不是协程，因为它是即时执行的而不是随时间进行的），以便在过场动画结束后让玛德琳能够再次移动。

```
function onBegin()
    disableMovement()
    run(16)
end

function onEnd()
    enableMovement()
end
```

Section TL;DR: TL;DR 部分:

Lua cutscenes are surrounded in a function called `onBegin()`. Some basic elementary functions include `say()`, `walk()`, `wait()`, `disableMovement()`, and `makeUnskippable()`. Once the `onBegin()` function is over, Lua will automatically jump to the `onEnd()` function, which can run some final code, but it cannot run coroutines (functions that "take time" to execute.)

Lua 过场动画被包含在一个名为 `onBegin()` 的函数中。一些基本的初始函数包括 `say()`、`walk()`、`wait()`、`disableMovement()` 和 `makeUnskippable()`。一旦 `onBegin()` 函数结束，Lua 将自动跳转到 `onEnd()` 函数，该函数可以运行一些最终代码，但它不能运行协程（执行需要“时间”的函数）。

Chapter 2: Loops and Statements

第 2 章：循环和语句

All our elementary functions work great, but when you start using a bunch of them, your cutscene can start looking like this:

我们所有的初始函数都运行得很好，但当您开始使用大量它们时，您的过场动画可能会开始看起来像这样：

```
function onBegin()
    wait(2.0)
    walkTo(620, false, 0.4)
    walk(-28, false, 0.4)
    walk(28, false, 0.4)
    walk(-28, false, 0.4)
    walk(28, false, 0.4)
    walk(-28, false, 0.4)
    walk(28, false, 0.4)
    ...
end
```

When you start copypasting the same function over and over, it's time to introduce a “loop” — a little bit of Lua you put around your code to make it run over and over again.

当你反复粘贴相同的函数时，就该引入“循环”——你可以在代码周围添加一小段 Lua，让它反复运行。

The `while (condition) do` loop is the most basic loop out there. When it's run for the first time, it will check if its condition is true. If it is, it will run the code inside of it. Once it has reached the end of the code inside of it, it will check the condition again. If it's still true, it will run the code inside again. Rinse and repeat until the condition is no longer true.

`while (condition) do` 循环是最基本的循环。当它第一次运行时，会检查条件是否为真。如果是，它会运行其中的代码。一旦到达代码的末尾，它会再次检查条件。如果仍然为真，它会再次运行其中的代码。重复此过程，直到条件不再为真。

```
local counter = 5
while counter > 0 do
    walk(28, false, 0.4)
    counter = counter - 1
end
```

There's a few new things being introduced there, so let's break them down one by one. The first thing we're doing is defining our own variable with the `local` keyword. Whenever you define a variable in your own Lua Cutscene, it's good practice to always use the `local` keyword before its name to limit its scope. Next, we give it the name of `counter` — we can actually name this

whatever we want, but I'm going to be using `counter` here because it's easy to understand.

这里介绍了一些新事物，让我们逐一分解。我们首先使用 `local` 关键字定义自己的变量。在 `Lua Cutscene` 中定义变量时，建议始终在变量名前使用 `local` 关键字来限制其作用域。接下来，我们给它命名为 `counter` —— 我们实际上可以随意命名，但在这里我将使用 `counter`，因为它易于理解。

We then give “counter” a value of 5. Variables in Lua can hold any data type — strings, ints, whatever. Every time the loop runs, we set “counter” to its current value minus one using an equals sign, effectively decrementing it by 1 every loop. Eventually, counter will no longer be greater than 0, and the loop will stop running.

然后我们将“counter”的值设为 5。在 Lua 中，变量可以存储任何数据类型——字符串、整数、等等。每次循环时，我们使用等号将“counter”设置为当前值减一，实际上每次循环都会将其减 1。最终，`counter` 将不再大于 0，循环将停止运行。

Conditional statements (like `counter > 0` in that example) are evaluated using “relational operators.” Going over them fully isn't in the scope of this tutorial, but in Lua you have the operators `and`, `not`, `or`, `>`, `<`, `==` (equals), `<=` (less than or equal to), `>=` (greater than or equal to), and `~=` (not equal). You can read about all of these operators [here](#) and [here](#). Conditional statements naturally lead to the “if statement:”

条件语句（如示例中的 `counter > 0`）使用“关系运算符”进行评估。在此教程中，全面介绍这些运算符不在范围内，但在 Lua 中有 `and`、`not`、`or`、`>`、`<`、`==`（等于）、`<=`（小于或等于）、`>=`（大于或等于）和 `~=`（不等于）这些运算符。你可以在这些地方阅读所有这些运算符的信息。条件语句自然地引出了“if 语句：”

```
if (counter > 0) then
    dash(1,0)
end
```

“If statements” will not repeat once they have been concluded like while loops. They run one time and one time only.

“if 语句”一旦执行完毕就不会重复，就像 `while` 循环一样。它们只运行一次，仅此一次。

The other main loop to know is the “numeric for.” Lua actually has 2 different kinds of for loops, but the other (the “generic for” — you can read about it [here](#)) is more advanced and not as relevant to Celeste. The “numeric for” handles the step of creating and incrementing / decrementing your

counter variable for you, which makes it very convenient.

另一个需要了解的主要循环是“数值 for 循环”。Lua 实际上有两种不同的 for 循环，但另一种（“通用 for 循环”——你可以在这里阅读相关信息）更高级，与 Celeste 的相关性不大。“数值 for 循环”会处理创建和递增/递减你的计数器变量的步骤，这使得它非常方便。

```
for counter=0,5,1 do
    dash(1,0)
end
```

As you can see, the statement has 3 parts:

如你所见，这个语句有三部分：

- The variable declaration. Here, we give our variable a name (I'm using counter, though i is the most common) and a starting value.

变量声明。在这里，我们给变量一个名称（我使用的是 counter，但 i 是最常见的）和初始值。

- The final value. This loop will continue until the variable reaches this value.

最终值。这个循环会持续到变量达到这个值。

- The delta value. This is how much the variable will change by each time the loop runs. I have it set to 1 in the example, so every time the loop repeats, counter will increment by 1.

增量值。这是每次循环时变量变化的量。在示例中我将其设置为 1，所以每次循环重复时，counter 会自增 1。

Generically, this looks like this:

通用形式如下：

```
for i=initial,final,delta do
    ...
end
```

This does the same thing as the while loop, but in an even more condensed form. So if we want to walk 28 pixels 30 times over, like in the example earlier, we can just write this:

这和 while 循环做相同的事情，但形式更加简洁。所以如果我们想像示例中那样走 30 次，每次 28 像素，我们只需这样写：

```
function onBegin()
    for i=0,30,1 do
        walk(28, false, 0.4)
    end
end
```

Section TL;DR TL;DR 部分

Loops like `while do` and `for do`, alongside statements like `if then`, allow you to add conditional execution to your cutscene and reduce redundancy. Lua allows you to evaluate conditional statements using various relational operators, including `>` (greater than), `<` (less than), and `==` (equals to). If you ever end up copy-pasting the same line 30 times, you’re doing something wrong.

循环如 `while do` 和 `for do`，以及语句如 `if then`，允许您为您的过场动画添加条件执行并减少冗余。Lua 允许您使用各种关系运算符来评估条件语句，包括 `>`（大于）、`<`（小于）和 `==`（等于）。如果您发现自己重复粘贴同一行 30 次，那说明您做错了。

Chapter 3: Customizing your Madeline

第三章：定制你的玛德琳

Earlier I mentioned that the `walk()` function will automatically play the walking animation for you. What if we wanted to play our own animation? How would we go about doing that?

之前我提到 `walk()` 函数会自动为您播放行走动画。如果我们想播放自己的动画呢？我们该如何实现？

In short, you do this:

简而言之，您需要这样做：

```
-- Say something, then sit down.
function onBegin()
    player.DummyAutoAnimate = false -- Let us play non-default animations
    say("BOY_I_AM_SLEEPY") -- Say something
    player.Sprite:Play("sitDown") -- Sit down
end
```

The long answer: 详细解释：

In this case, Lua Cutscenes doesn’t provide us with any convenient pre-made function to play an animation. But it does let us access any aspect of Celeste’s own code, which we can use to do **roughly anything**. There’s a couple limitations — it’s not quite feasible to create a custom entity — but something like “playing an animation”, “flickering stylegrounds”, “making noise”, or

“loading in a new room” are all possible, though not always advisable.

在这种情况下，`Lua Cutscenes` 没有提供任何方便的预置函数来播放动画。但它确实允许我们访问 `Celeste` 自身的任何代码部分，我们可以利用这些代码来做任何事情。有一些限制——创建自定义实体并不可行——但像“播放动画”、“闪烁风格地面”、“制造噪音”或“加载新房间”之类的功能都是可能的，尽管并不总是推荐这样做。

From here, we can no longer rely on a little list of functions to mix together, and instead need to learn to read the game’s original code... or at least, a recreation of it from your legally-obtained installation, since the code isn’t distributed publicly online for obvious reasons. `Celeste` is written in C#, which has a variety of useful “decompiler” tools made exactly for this task; try opening `Celeste.dll` using ILSpy (my pick) or DNSpy.

从这里开始，我们不能再依赖一小列表函数来混合使用，而是需要学习阅读游戏的原版代码.....或者至少，从你合法获取的安装版中重新创建它的代码，因为出于明显的原因，代码并没有公开发布在网上。`Celeste` 是用 C# 编写的，有各种有用的“反编译器”工具专门为此任务制作；尝试使用 `ILSpy`（我的选择）或 `DNSpy` 打开 `Celeste.dll`。

There’s a lot going on, and at first, it won’t always be obvious how to achieve a particular goal. But since we want to change how the player looks, let’s start by searching for “Player”. You’ll find a `Player.cs` file, and inside, a `class Player` !

有很多事情在进行中，一开始可能不会明显知道如何实现特定目标。但由于我们想改变玩家的外观，让我们从搜索“Player”开始。你会找到一个 `Player.cs` 文件，里面有一个 `class Player` !

We won’t dwell on the ins-and-outs of C# — for now, anyway — but in short, classes are ways of organizing code. Each class is two things at once:

我们不会深入探讨 C# 的细节——至少目前不会——简而言之，类是组织代码的方式。每个类同时具有两个作用：

- A blueprint for objects. The `Player` class describes what “a player” is, the `Sprite` class describes what “a sprite” is, the `Entity` class describes the basic details shared by all entities. Variables and functions — which in this context are called “fields” and “methods” — describe attributes that an object can have, and things they can do. For example, the `twoDashes` field of `Refill` stores whether any particular refill is a double dash refill, and will have different values for different refills. “Calling” the `Destroy` method of `CrystalStaticSpinner` (that’s how we refer to running code inside a specific method, and what spinners are called) can be used to destroy a spinner and make it explode into shards before disappearing. Naturally, all of these can only be done in the context of a particular object (“what am I looking at or modifying”), called the “receiver”, and

this is the default meaning of code in C#.

对象的设计蓝图。 `Player` 类描述了“玩家”是什么， `Sprite` 类描述了“精灵”是什么， `Entity` 类描述了所有实体共享的基本细节。变量和函数——在这个语境中被称为“字段”和“方法”——描述了对象可以拥有的属性以及它们能做的事情。例如， `Refill` 的 `twoDashes` 字段存储了任何特定填充是否是双短划线填充，并且对于不同的填充会有不同的值。“调用”

`CrystalStaticSpinner` 的 `Destroy` 方法（这是指在特定方法内部运行代码的方式，也是旋转器的调用方式）可以用来销毁旋转器，使其在消失前爆炸成碎片。当然，所有这些操作都只能在特定对象的上下文中进行（“我正在查看或修改什么”），这个对象被称为“接收者”，这也是 C# 中代码的默认含义。

```
//This is a class! Note it has both fields and methods.
public class Player
{ ...
    //This is a field! Note it has no attached code and is just a variable.
    private int lastDashes;
    ...
    //This is a method! Note that it does have attached code, namely
    //all this "vector2" stuff. We'll get into that way later.
    private void BoostEnd()
    {
        Vector2 vector = (boostTarget - base.Collider.Center).Floor();
        MoveToX(vector.X);
        MoveToY(vector.Y);
    }
    ...
}
```

- A place to put code. Not all C# code is attached to a particular object; for example, why would `sin(x)`, or the speedrun timer's current time, or the particular shade of blue you use for something need any context? These are more similar to the Lua code you've been writing so far, just organized into multiple files with descriptive names for better organization. (There's slightly more to it in C#, of course, but this is all you should concern yourself with.) **This is not the default in C#; context-free things like this are marked as "static".**

一个放置代码的地方。并非所有 C# 代码都与特定对象关联：例如，为什么 `sin(x)`、速通计时器的当前时间，或你为某物使用的特定蓝色需要任何上下文？这些更类似于你到目前为止编写的 Lua 代码，只是组织在多个具有描述性名称的文件中，以便更好地组织。（当然，C# 中还有更多细节，但这就是你应该关注的内容。）在 C# 中这不是默认设置：像这样无需上下文的东西被标记为“static”。

```
public class Player
{
    //This field is static!
    private static int diedInGBJ;
    ...
    //This method is not static - in other words, it's an "instance."
    private void WallJump(int dir)
    {
```

```
orig_WallJump(dir);  
}  
...  
}
```

An important thing to note is that fields and methods tell you what kind of data they have in them (e.g., a string or an integer).

需要注意的是，字段和方法会告诉你它们包含什么类型的数据（例如，字符串或整数）。

```
//We see here that this field is holding an integer, or int.  
private static int diedInGBJ;
```

Lua **doesn't** tell you this, but knowing it is very useful for both reverse-engineering and building bigger projects than one cutscene. You might also notice that the naming convention is different while looking through the decompilation. Names still can't use spaces, but the first letter is also usually capitalized on everything. Don't worry too much about these conventions.

Lua 不会告诉你这些，但了解这些对于逆向工程和构建比一个过场动画更大的项目非常有用。你可能会注意到，在反编译过程中命名约定有所不同。名称仍然不能使用空格，但通常所有单词的首字母也大写。不必过于担心这些约定。

You'll also notice the word `private` in front of some fields or methods.

你还会注意到一些字段或方法前面有 `private` 这个词。

```
//This field controls whether a strawberry seed has a ghost sprite or not!  
private bool ghost;  
//This method controls what happens when a player touches a strawberry seed!  
private void OnPlayer(Player player) { ... }
```

This means that the method is, well, private. **It can't be easily used or accessed.** Way, way, way down the line we'll get into how you can do that anyway, but for now if you see the word `private` in front of a method or field you want to use in a cutscene, assume you're out of luck. Fortunately, most things you'll casually be using like camera or player movement have the word `public` attached to them instead. You can use or access public methods or fields with no effort whatsoever.

这意味着这个方法是私有的。它不容易被使用或访问。在很久以后，我们会讲到如何无论如何使用它，但目前如果你在剪辑中看到某个方法或字段前有 `private` 字样，就假定你运气不好。幸运的是，大多数你随意使用的东西，如相机或玩家移动，都有 `public` 附加在它们上面。你可以毫不费力地使用或访问公共方法或字段。

Back to animation! The `Player` class has a convenient field called `Sprite`, which stores a `PlayerSprite`. You might correctly guess that this controls what's displayed on the screen, but searching around in `PlayerSprite` itself gives you nothing useful for playing an animation; what gives? Well, it's a sprite in the same way that the player is an entity, without either of those having to redefine all the basics of drawing or physics; they're based on, or "inherit from", the simpler classes `Sprite` and `Entity`. That's what the `class PlayerSprite : Sprite` at the top of the file means: it is a `Sprite`, and more.

回到动画！`Player` 类有一个方便的字段 `Sprite`，它存储了一个 `PlayerSprite`。你可能正确地猜测这控制着屏幕上显示的内容，但在 `PlayerSprite` 本身中搜索却什么有用的东西都没有来播放动画；这是怎么回事？嗯，它是一个精灵，就像玩家是一个实体一样，而它们都不需要重新定义所有基本的绘制或物理；它们基于或“继承自”更简单的类 `Sprite` 和 `Entity`。这就是文件顶部 `class PlayerSprite : Sprite` 的意思：它是一个 `Sprite`，并且更多。

Looking in `Sprite.cs`, you'll find a nice looking `Play` method:

在 `Sprite.cs` 中，你会找到一个漂亮的 `Play` 方法：

```
/// <summary>Play an animation stored in this sprite.</summary>
/// <param name="id">The animation to play.</param>
/// <param name="restart">Whether to restart the animation if it is already play
/// <param name="randomizeFrame">Whether to randomize the starting frame and ani
public void Play(string id, bool restart = false, bool randomizeFrame = false) {
```

(The `///` is for writing comments that describe or document definitions, like a more specialized version of the two dashes Lua uses. C# has `//` for general-purpose comments, but you will find very few in decompiled code. This comment formatting tangent only applies to C#; it's included here to make your decompiled code reading experience easier.)

(The `///` 用于编写描述或记录定义的注释，类似于 Lua 中双破折号的一种更专业的版本。C# 使用 `//` 进行通用注释，但在反编译代码中你会发现很少使用。这个注释格式的小插曲仅适用于 C#；它被包含在这里是为了让你的反编译代码阅读体验更轻松。)

So all we need to do is figure out what to put in for those different inputs, and how exactly to write it out in Lua. In short (or in long, really):

所以我们需要做的一切就是弄清楚如何为这些不同的输入提供内容，以及如何在 Lua 中精确地写出它们。简而言之（或者长篇大论地说）：

- Animation IDs come from `Sprites.xml` files, the same ones you might have edited to override vanilla animations in maps already. In the vanilla version, the `<player>` section contains an `<Anim id="sitDown">`, which is

where `sitDown` from earlier comes from.

动画 ID 来自 `Sprites.xml` 文件，这些文件可能你已经编辑过，用于在地图中覆盖原版动画。在原版版本中，`<player>` 部分包含一个 `<Anim id="sitDown">`，这就是之前提到的 `sitDown` 的来源。

```
...  
<Anim id="sitDown" path="sitDown" delay="0.1" /> #Hey, there it is!  
...
```

- Looking back at the `Play()` method of `Sprite`, it actually has 3 parameters, not just 1 for the animation ID. However, both of the other inputs have default values (both `false`), so we don't even need to specify them at all. But we could choose to, which might be appropriate for a different animation (especially one that loops.)

回顾一下 `Play()` 的 `Sprite` 方法，它实际上有 3 个参数，而不仅仅是 1 个动画 ID 参数。然而，其他两个输入都有默认值（都是 `false`），所以我们甚至不需要指定它们。但我们也可以选择指定它们，这可能适用于不同的动画（尤其是循环动画）。

```
player.Sprite:Play("sitDown") -- Good for a one time anim  
player.Sprite:Play("launch", true) -- Good for a looping anim
```

- This method is `public`, so we don't have to do anything weird in that regard. It also doesn't have the `static` keyword attached to it, meaning it's a normal method that we refer to as an instance method. **To run instance methods in Lua, we need to use a colon instead of the traditional period.**

这种方法是 `public`，所以我们在这方面不需要做任何奇怪的事情。它也没有 `static` 关键字附加在上面，这意味着它是一个我们称之为实例方法的普通方法。在 Lua 中运行实例方法时，我们需要使用冒号而不是传统的点。

```
-- Play is a non-static / instance method, so here we use a colon  
player.Sprite:Play("sitDown")  
-- SetSpeedrunClock is a static method in the MenuOptions class,  
-- so here we would use a dot  
MenuOptions.SetSpeedrunClock(10000)
```

- But first, we need to get the sprite. **Just like static methods, we can access fields with a dot**, and conveniently Lua Cutscenes provides us with a `player` variable holding the player without any definition work on our

part. (We'll get to custom definitions later.) So `player.Sprite` is sufficient.

但首先，我们需要获取精灵。就像静态方法一样，我们可以用点来访问字段，而且方便的是 `Lua Cutscenes` 提供了一个 `player` 变量来持有玩家，而无需我们进行任何定义工作。（我们稍后会谈到自定义定义。）所以 `player.Sprite` 就足够了。

- Finally, the player tries to play the appropriate animation during cutscenes automatically, which is why `walk()` works properly, but also stops our custom ones from running (for more than a single frame). This behavior is controlled by the `DummyAutoAnimate` field, which we turn off here. Don't forget to turn it back on (to `true`) if you want to use `walk()` (or anything else with its own animations)!

最后，玩家在过场动画期间会自动播放适当的动画，这就是为什么 `walk()` 能正常工作，但也阻止了我们的自定义动画运行（超过单帧）。这种行为由 `DummyAutoAnimate` 字段控制，在这里我们将其关闭。如果你想要使用 `walk()`（或任何具有自己动画的东西），别忘了将其重新打开（设置为 `true`）。

```
-- Since this is a field, we use the dot to access and edit it
player.DummyAutoAnimate = false
```

And that's where our code from before comes from. This was a short summary of the whole process, but thankfully, this is about as complex as it gets. The `Player` class is possibly the most complex one in the game, with hundreds of fields and 5 thousand lines, which you will never need to understand all at once. Common use-cases have already been figured out, and you can always ask for help in figuring out something entirely new. And it gets much, much easier to read and navigate the code with time and practice; and once you figure it out, you're no longer limited by pre-made functions and can do anything you want.

这就是我们之前代码的来源。这是整个过程的简短总结，但幸运的是，这基本上就是它的全部复杂性。`Player` 类可能是游戏中最为复杂的类之一，拥有数百个字段和 5000 行代码，你永远不需要一次性理解所有内容。常见的使用场景已经得到解决，如果你需要探索全新的东西，随时可以寻求帮助。随着时间的推移和实践，阅读和导航代码会变得容易得多；一旦你弄明白了，你就不再受限于预制的函数，可以随心所欲地做任何事。

```
-- Say something, then sit down.
function onBegin()
    player.DummyAutoAnimate = false
    say("BOY_I_AM_SLEEPY")
    player.Sprite:Play("sitDown")
end
```

Lua Cutscenes have the ability to access any code from the original C# Celeste codebase. Classes in C# contain fields, which are variables that apply to the entire class, and methods, which are little functions that run specific bits of code. To access a C# player method in Lua Cutscenes, just prefix the method call with `player`. By default, methods in C# are referred to as instance methods and are accessed in Lua Cutscenes with a colon. Static methods and fields are accessed with a period. If a field or method is private, it cannot be accessed easily.

Lua Cutscenes 能够访问原始 C# Celeste 代码库中的任何代码。C# 中的类包含字段，这些字段是适用于整个类的变量，以及方法，这些方法是运行特定代码片段的小函数。要在 Lua Cutscenes 中访问 C# 玩家方法，只需在方法调用前加上 `player`。默认情况下，C# 中的方法被称为实例方法，在 Lua Cutscenes 中通过冒号访问。静态方法和字段通过点号访问。如果字段或方法是私有的，则无法轻松访问。

Chapter 4: Camera 第四章：摄像机

So far we've looked into moving the player, both in terms of animating them and moving them from place to place. Let's expand our scope!

到目前为止，我们已经研究了如何移动玩家，包括动画效果和在不同位置之间移动。让我们扩展一下我们的范围！

Often in a cutscene you'll want to move or zoom the camera. To rip this bandaaid off: while you can zoom out, it looks abysmal and should not be used. Otherwise, all camera movement is viable, and adds the perfect cinematic flair to your cutscene.

在过场动画中，你通常需要移动或缩放摄像机。直截了当地说：虽然可以缩放，但效果非常糟糕，不建议使用。除此之外，所有摄像机移动都是可行的，并且能为你的过场动画增添完美的电影感。

The camera in Celeste is (mostly) handled by the `Level` class. In the earlier example with sprite animation, Lua Cutscenes graciously provided a pre-made `player` variable for us. It doesn't do that here, so we have to make our own to hold the current `Level`.

Celeste 中的摄像机（主要是）由 `Level` 类处理。在之前的精灵动画示例中，Lua Cutscenes 为我们提供了一个现成的 `player` 变量。但在这里，它没有这样做，所以我们必须创建自己的变量来保存当前的 `Level`。

```
function onBegin()
    local level = getLevel()
    ...
end
```

As you can tell by the parentheses at the end of `getLevel()`, it's a method. This code introduces a new concept: when a method runs, it can give a value

back to the code that called it. What the value is depends on the specific method, and this is called a “return” value. “Value” is a bit reductive; a method can return anything, whether it be a string, integer, an instance of a class, or more. You’re able to tell what a method returns by reading its header in the decompiled code.

从 `getLevel()` 末尾的括号可以看出，这是一个方法。这段代码引入了一个新概念：当方法运行时，它可以向调用它的代码返回一个值。返回的值取决于具体的方法，这被称为“返回值”。“值”这个词有点简化；一个方法可以返回任何东西，无论是字符串、整数、类的实例，还是更多。你可以通过阅读反编译代码中的方法头部来知道一个方法返回什么。

```
private bool DreamDashCheck()
{
    if (Inventory.DreamDash && DashAttacking && ...
}
```

Here, this method (used by the `Player` class to check if the player is in a dream block or not) will return a “bool” (aka boolean, aka true or false) value.

这里，这种方法（由 `Player` 类用来检查玩家是否处于梦境区块）将返回一个“bool”（即布尔值，即 `true` 或 `false`）值。

Back to our `getLevel()` example, that method returns the current instance of the `Level` class. In simpler terms, it gives us access to the code that’s controlling the camera and a bunch more. By setting it to a variable, we can access it and manipulate it as much as we want.

回到我们的 `getLevel()` 示例，该方法返回 `Level` 类的当前实例。用更简单的话来说，它让我们能够访问控制摄像头的代码以及其他更多内容。通过将其赋值给一个变量，我们就可以访问并按需操作它。

That was all pretty confusing, and all you really need to know is that before doing any camera stuff you need to create a local variable of any name and set its value to `getLevel()`. However, this is only half of manipulating the camera; for the other half, we have to return to the coroutines from earlier.

这一切都相当令人困惑，而你需要真正了解的是，在进行任何摄像头操作之前，你需要创建一个名为任何名称的局部变量，并将其值设置为 `getLevel()`。然而，这仅仅是操控摄像头的半部分；另一半，我们需要回到之前的协程。

As defined before, a coroutine as a function that “takes time” to execute. You can tell if a function is a coroutine by checking its return type. Above, we saw a function that returns a “bool” — if a function returns an

“IEnumerator,” it’s a coroutine.

如之前定义的，协程是一个“耗时”执行的函数。你可以通过检查其返回类型来判断一个函数是否是协程。之前，我们看到一个返回“bool”的函数——如果一个函数返回“IEnumerator”，它就是一个协程。

```
public IEnumerator DummyWalkTo(...)
{
    StateMachine.State = 11;
    ...
}
```

This is actually the walk function from earlier! The reason why it looks a little different (`DummyWalkTo()` instead of `walkTo()`) is because the developer of Lua Cutscenes was helpful and allowed you to run it without typing what I call the “coroutine wrapper.” Normally, whenever you call a coroutine, it must be wrapped in this specific formatting:

这实际上是之前提到的行走函数！它看起来有些不同（使用 `DummyWalkTo()` 而不是 `walkTo()` ）的原因是 Lua Cutscenes 的开发者很友好，允许你运行它而无需输入我所说的“协程包装器”。通常，每次调用协程时，都必须用这种特定格式进行包装：

```
function onBegin()
    coroutine.yield(...)
end
```

You’ve probably guessed by now that all camera movement is done through coroutines, and you would be right. Since we don’t have helpful shortcuts from the Lua Cutscene dev here, you’re going to need to put your camera code inside that coroutine wrapper. Besides that, it’s just more of what we did above.

现在你大概已经猜到所有摄像机移动都是通过协程完成的，而且你是对的。由于这里没有 Lua Cutscene 开发者提供的友好快捷方式，你需要将你的摄像机代码放在那个协程包装器里面。除此之外，它只是我们之前所做内容的更多部分。

`Level` contains several camera-related methods, the most basic being `ZoomTo()`. It’s an instance method, meaning to access it, we use a colon instead of a dot.

`Level` 包含多个与摄像机相关的函数，最基本的是 `ZoomTo()`。它是一个实例方法，这意味着要访问它，我们使用冒号而不是点。

`ZoomTo()` has 3 parameters. We’re going to go through them in sequence — **note that you need ALL of them in order for the method to work, even if in the code blocks I’m not showing them all yet.** Just hold tight and we’ll get

through them all! In order, they are:

`ZoomTo()` 有三个参数。我们将按顺序逐一介绍它们——请注意，即使在我尚未全部展示的代码块中，为了让这个方法工作，你需要所有这些参数。耐心一点，我们会逐一介绍它们！按顺序，它们是：

- `screenSpaceFocusPoint` — this point is given in the form of a `vector2`, which is something we haven't talked about yet. A `vector2` is a way of representing a point in 2D space, or in other words, a fancy way to give X and Y coordinates for the camera to zoom to. The syntax for a `vector2` point is `vector2(x, y)`.

`screenSpaceFocusPoint` — 这一点以 `vector2` 的形式给出，这是我们还没有讨论过的事情。`vector2` 是在二维空间中表示一个点的方法，换句话说，是一种为相机缩放提供 X 和 Y 坐标的优雅方式。`vector2` 点的语法是 `vector2(x, y)`。

```
function onBegin()
  -- Remember, to use the Level functions, we need to get it first
  local level = getLevel()
  -- Use your coroutine wrapper!
  -- Zoom to the point 150, 150
  coroutine.yield(level:ZoomTo(vector2(150, 150)))
end
```

`ZoomTo()` specifically is weird and uses a different coordinate system than every other camera function. The screen in Celeste is 320 x 180 pixels across; `ZoomTo()` wants coordinates relative to the screen. This means that putting 160, 90 as our coordinates will zoom in directly to the center. (Remember that increasing Y moves the camera *down* in Celeste, not up like on the Cartesian plane.)

`ZoomTo()` 特别的是奇怪，它使用与其他所有相机功能不同的坐标系。Celeste 中的屏幕宽 320 x 180 像素；`ZoomTo()` 需要相对于屏幕的坐标。这意味着将坐标设置为 160, 90 将直接缩放到中心。（记住在 Celeste 中，增加 Y 会将相机向下移动，而不是像笛卡尔平面那样向上移动。）

```
function onBegin()
  local level = getLevel()
  -- Zoom to 40 pixels below the center of the screen
  coroutine.yield(level:ZoomTo(vector2(160, 130)))
end
```

Back to the other parameters:

回到其他参数：

- `zoom` — this is given in the form of a `float`. A floating point number, or a `float`, is one way a computer represents decimals. Basically, this

represents how much the camera should zoom in and it can take both integer and decimal amounts.

`zoom` — 这以 `float` 的形式给出。浮点数，或 `float`，是计算机表示小数的一种方式。基本上，这表示相机应该缩放多少，它可以接受整数和小数。

```
function onBegin()
    local level = getLevel()
    -- Zoom in 2x, focus on the point 150, 150
    coroutine.yield(level:ZoomTo(vector2(150, 150), 2))
end
```

- `duration` — this is also given as a float. It represents how long the zoom should take, in seconds. Pretty simple.

`duration` — 这个也以浮点数形式给出。它表示缩放应该持续多长时间，以秒为单位。相当简单。

```
function onBegin()
    local level = getLevel()
    -- Over the next 3.5 seconds, zoom in 2x, focusing on the point 150, 150
    coroutine.yield(level:ZoomTo(vector2(150, 150), 2, 3.5))
end
```

`Level` provides some other smooth zoom functions:

`Level` 提供了一些其他的平滑缩放功能：

- `ZoomBack(duration)` , which resets the camera to its default state over a set duration

`ZoomBack(duration)`，在设定的持续时间后将相机重置为其默认状态

- `ZoomAcross(screenSpaceFocusPoint, zoom, duration)` , which should be used instead of `ZoomTo` if the camera is already zoomed in. This will make the transition much smoother.

`ZoomAcross(screenSpaceFocusPoint, zoom, duration)`，如果相机已经缩放，则应使用这个代替 `ZoomTo`。这将使过渡更加平滑。

... and some less smooth camera functions:

...还有一些不太流畅的相机功能：

- `ZoomSnap(screenSpaceFocusPoint, zoom)` , which immediately sets the camera to focus on the specific point with the specific zoom

`ZoomSnap(screenSpaceFocusPoint, zoom)`，立即将相机对准特定点并使用特定缩放

- `ResetZoom()` , which immediately resets the camera back to its default state

`ResetZoom()` , 立即将相机重置为其默认状态

Notably, these last two functions **are not coroutines** since they happen instantaneously. This means you do not need to use the coroutine wrapper and they can be used in `onEnd()` without any crashes.

值得注意的是，最后这两个功能不是协程，因为它们是即时执行的。这意味着你不需要使用协程包装器，并且它们可以在 `onEnd()` 中使用而不会崩溃。

```
function onEnd()
    coroutine.yield(level:ZoomTo(vector2(150, 150), 1, 3.5)) -- bad, will crash
    coroutine.yield(level:ResetZoom()) -- bad, using wrapper with non coroutine
    level:ResetZoom() -- good :)
end
```

With these functions, you should be set for zooming the camera, but what about basic movement? For that, we need to access the `CutsceneEntity` class instead of the `Level` class, as `Level` can **only** handle zooms. Just like `player`, this class is provided for us by the Lua Cutscene dev and is created behind the scenes whenever a cutscene begins to run.

有了这些函数，你应该已经可以缩放相机了，但基本移动呢？为此，我们需要访问 `CutsceneEntity` 类而不是 `Level` 类，因为 `Level` 只能处理缩放。就像 `player` 一样，这个类是由 Lua Cutscene 开发者提供的，在每次过场动画开始运行时都会在后台创建。

The `CutsceneEntity` class has a method called `CameraTo`:

`CutsceneEntity` 类有一个名为 `CameraTo` 的方法：

```
/// <summary> Ease the level's camera towards a point </summary>
/// <param name="target">The point to move the camera to.</param>
/// <param name="duration">Duration of the ease.</param>
/// <param name="ease">Type of ease (Default: CubeInOut).</param>
/// <param name="delay">Delay before beginning the ease.</param>
public static IEnumerator CameraTo(Vector2 target, float duration, Ease.Easer ea
{ ... }
```

As we can see from the method header, this method is both static and a coroutine, so when we call it we're going to need to use the coroutine wrapper and a period instead of a colon. It takes 4 parameters — `target`, `duration`, `ease`, and `delay`, but `ease` and `delay` already have default values, so we only need to give the method values for `target` and `duration` inside its

parentheses when we use it.

从方法头可以看出，这个方法是静态的也是协程，所以当我们调用它时，我们需要使用协程包装器，并用点号代替冒号。它接受 4 个参数—— `target` 、 `duration` 、 `ease` 和 `delay` ，但 `ease` 和 `delay` 已经有了默认值，所以我们只需要在使用时在其括号内给出 `target` 和 `duration` 的值。

```
function onBegin()  
  -- Remember to include the "celeste" before the CutsceneEntity bit,  
  -- since we need to specify where we're getting CutsceneEntity from  
  coroutine.yield(celeste.CutsceneEntity.CameraTo(target, duration))  
end
```

`target` , like we saw earlier, is a `Vector2` value. Unlike `ZoomTo()` , however, this function takes coordinates relative to the entire map, not just the screen. These coordinates are much harder to intuit than screen coordinates — it is highly recommended to install the mod [CelesteTas](#) to help you find them. If you open the debug console (via `.` or the backtick ```) and then click anywhere on the screen, you'll find a small menu has opened in the top left of the screen which displays your mouse's position in “world” coordinates. This is the value you want.

`target` ，正如我们之前看到的，是一个 `Vector2` 值。然而，与 `ZoomTo()` 不同的是，这个函数使用的是相对于整个地图的坐标，而不是屏幕坐标。这些坐标比屏幕坐标更难直观理解——强烈建议安装模组 [CelesteTas](#) 来帮助你找到它们。如果你打开调试控制台（通过 `.` 或反引号 ```），然后在屏幕上任意位置点击，你会发现屏幕左上角出现了一个小菜单，显示鼠标在“世界”坐标中的位置。这就是你需要的值。

Duration is the same as earlier, taking any positive real number value.

持续时间与之前相同，可以取任何正实数值。

```
function onBegin()  
  -- Move the camera to focus on (150, 150) over the course of 3 seconds.  
  coroutine.yield(celeste.CutsceneEntity.CameraTo(vector2(150, 150), 3))  
end
```

So let's put it all together:

那么让我们把所有内容整合起来：

```
function onBegin()  
  -- Store the current Level instance  
  local level = getLevel()  
  -- Zoom in 2x, focusing on the point (150, 150), over the next 3.5 seconds  
  coroutine.yield(level.ZoomTo(vector2(150, 150), 2, 3.5))  
  -- While maintaining this zoom level, focus on the point (300, 300) over the  
  -- next 3 seconds  
  coroutine.yield(celeste.CutsceneEntity.CameraTo(vector2(300, 300), 3))  
end
```

```
-- While still focusing on the point (300, 300), reset back to 1x zoom level
level:ResetZoom()
end
```

Section TL;DR: 简明扼要:

We can create local variables that hold instances of classes in order to manipulate them as we desire. To zoom the camera, we first need to get the `Level` instance, then we can use the various `Zoom` functions with coroutines. **Camera movement is done by using the `CameraTo` coroutine of the `CutsceneEntity` class.** A method can be identified as a coroutine if it returns an `IEnumerator` value.

我们可以创建局部变量来存储类的实例，以便按需操作它们。要缩放相机，我们首先需要获取 `Level` 实例，然后可以使用协程配合 `Zoom` 的各种函数。相机移动是通过使用 `CutsceneEntity` 类的 `CameraTo` 协程来完成的。如果方法返回 `IEnumerator` 值，则可以将其识别为协程。

Chapter 5: Parallel Execution

第五章：并行执行

So far, everything we've done has been run in sequence — line 1 runs, then line 2, then line 3, etc. What if we wanted to run something in parallel, like zooming and moving the camera at the same time?

到目前为止，我们做的一切都是按顺序执行的——第 1 行运行，然后第 2 行，然后第 3 行，等等。如果我们想并行运行某些操作，比如同时缩放和移动相机呢？

This is where Lua Cutscenes gets kind of ugly. Previously, we had one simple coroutine wrapper for all of our time-taking-function needs. Now, we're going to have 2 different layers — let's take a look at Monocle Coroutines.

这就是 Lua Cutscenes 变得有些丑陋的地方。之前，我们有一个简单的协程包装器来处理所有耗时的函数需求。现在，我们将有两个不同的层——让我们来看看 Monocle Coroutines。

```
local monocle = require("#monocle")
local coroutineEntity = monocle.Entity()

function onBegin()
    local level = getLevel()
    level:add(coroutineEntity)
    coroutineEntity:add(monocle.Coroutine(celeste.CutsceneEntity.CameraTo(vector2(
    routineEntity:add(monocle.Coroutine(celeste.CutsceneEntity.CameraTo(vector2(300,
end
```

So that's a lot. If you don't want to learn how this works on a technical level, this is the one section where I'd recommend skipping straight to the TL;DR.

If you're cool, though, let's take a look at those two top lines together first.

这有很多内容。如果你不想从技术层面学习它是如何工作的，这是唯一建议直接跳到 TL;DR 的部分。不过，如果你愿意，让我们先一起看看这两行顶部的代码。

Previously, whenever we got a method from a class, it's been coming from the base Celeste codebase. The Lua Cutsценe dev was very nice and implicitly “required” Celeste for us, so we didn't have to have a line like this at the top of all of our cutscenes.

之前，我们从一个类中获取方法时，这些方法一直来自基础 Celeste 代码库。Lua Cutsценe 开发者非常友好，隐式地“要求”了 Celeste，所以我们不需要在所有场景的顶部添加这样的行。

```
local celeste = require("#celeste")
```

Another implicitly “required” thing was `vector2`. This is actually a part of the “XNA framework” that Celeste uses behind the scenes, but we're still allowed to use it without adding a `require` statement at the top of our file due to the dev being nice. The Monocle engine is another behind-the-scenes game dev thing used by Celeste, but in contrast to `vector2`, it is *not* implicitly required. If we want to use anything from it, we have to add our own `require` statement.

另一个隐式“要求”的东西是 `vector2`。这实际上是 Celeste 在后台使用的“XNA 框架”的一部分，但由于开发者很友好，我们仍然可以在文件顶部不添加 `require` 语句来使用它。Monocle 引擎是 Celeste 在后台使用的另一个游戏开发工具，但与 `vector2` 不同，它不是隐式要求的。如果我们想使用它提供的任何东西，我们必须添加自己的 `require` 语句。

Thankfully, requiring Monocle — and things in general — is very simple. You just get their class name from the decomposition and throw it inside a local variable. If it comes from C#, which 99% of things you're going to ever use will, you need to prefix it with a hashtag / pound sign.

幸运的是，要求 Monocle 以及一般的东西非常简单。你只需要从分解中获取它们的类名，并将其放入一个局部变量中。如果它来自 C#，而你要使用的 99% 的东西都会来自 C#，你需要用井号/哈希符号作为前缀。

```
-- This is what the implicitly required lines would look like, if we
-- needed to require them ourselves
local celeste = require("#celeste")
local vector2 = require("#microsoft.xna.framework.vector2")
-- This is how you require a non-implicitly included class
local monocle = require("#monocle")
-- You can even require subclasses!
local calc = require("#monocle.calc")
```

The next line should be more familiar. All we're doing is creating a local variable and telling it to hold an instance of the `Entity` class from Monocle (we can do this now that we've required it — remember this is still running in sequence). It's basically the same thing as storing `getLevel()` from earlier.

下一行应该更熟悉。我们只是创建一个局部变量，并告诉它持有来自 Monocle 的 `Entity` 类的实例（我们现在可以这样做——记住这仍然按顺序运行）。这基本上和之前存储 `getLevel()` 一样。

```
-- Get Monocle so we can use it
local monocle = require("#monocle")
-- Store an instance of a Monocle entity
-- It's a surprise tool that will help us later
local coroutineEntity = monocle.Entity()
```

You may have noticed we're doing these local variable declarations outside of the main `onBegin()` function, whereas for `Level` we put it inside. This is because these two things — Monocle and a new `Entity` from Monocle — do not change based on what room we're in. `Level`, though, will be different depending on what room the cutscene starts in, so we need to wait to declare it until the cutscene begins. If you're unsure whether the thing you're storing will change based on the current state of the game, just put it inside the `onBegin()` function to be safe. It's good practice to find out and put it either inside or outside based on what's appropriate, but it won't crash and burn or anything if you just put it inside regardless.

你可能已经注意到，我们是在主 `onBegin()` 函数外部做这些局部变量声明，而对于 `Level` 我们把它放在里面。这是因为这两件事——Monocle 和一个新的 `Entity` 来自 Monocle——不会根据我们所在的房间而改变。`Level`，然而，会根据过场动画开始的房间而不同，所以我们需要等到过场动画开始时再声明它。如果你不确定你存储的东西是否会根据游戏的当前状态而改变，就把它放在 `onBegin()` 函数里面会更安全。好的做法是弄清楚并根据适当的情况把它放在里面或外面，但如果你不管三七二十一都把它放在里面，也不会崩溃或烧毁。

Next, we need to get and store `Level` just like we've been doing. So far, so good. Surely this is not foreshadowing that things are about to get a lot worse.

接下来，我们需要像我们一直做的那样获取和存储 `Level`。到目前为止，一切都很好。当然，这不会预示着事情即将变得更糟。

```
-- Get Monocle so we can use it
local monocle = require("#monocle")
-- Store an instance of a Monocle entity
local coroutineEntity = monocle.Entity()

function onBegin()
    -- Store the current Level
```

```
local level = getLevel()
end
```

So now things get worse. What we need to do is “add” the `coroutineEntity` we created earlier to the “scene.” To understand what this means, let’s look at the `Level` class in the decomposition.

现在情况变得更糟了。我们需要做的是将我们之前创建的 `coroutineEntity` 添加到“场景”中。为了理解这意味着什么，让我们看看分解中的 `Level` 类。

```
public class Level : Scene, IOverlayHandler
{ ... }
```

This line is the “class definition,” and it’s at the very top of the `Level.cs` file. The colon next to “Level” indicates that it “extends” 2 other classes, the important one being `Scene`. To “extend” a class means that you get all of the code (fields and methods) from the class you’re extending by default without having to write it all again. Since `Level` gets everything from `Scene`, let’s look at `Scene` now.

这一行是“类定义”，它位于 `Level.cs` 文件的顶部。紧挨着“Level”的冒号表示它“继承”了2个其他类，其中重要的是 `Scene`。要“继承”一个类意味着你可以默认获得你所继承类的所有代码（字段和方法），而无需再次编写所有内容。由于 `Level` 从 `Scene` 获得所有内容，我们现在来看看 `Scene`。

```
/// <summary>
/// Shortcut function for adding an Entity to the Scene's Entities list.
/// </summary>
/// <param name="entity">The Entity to add</param>
public void Add(Entity entity)
{ ... }
```

Very helpfully, the `Scene` provides us an easy method to add our entity. Again, `Level` gets everything from `Scene`, so we just need to call its `Add()` method with our `coroutineEntity` as the parameter. Let’s look at our cutscene again.

非常方便的是，`Scene` 提供了一种简单的方法来添加我们的实体。同样，`Level` 从 `Scene` 获得所有内容，所以我们只需要用我们的 `coroutineEntity` 作为参数调用它的 `Add()` 方法。让我们再看看我们的过场动画。

```
-- Get monocle
local monocle = require("#monocle")
-- Create a Monocle entity we can use to create coroutines later
local coroutineEntity = monocle.Entity()
function onBegin()
  -- Get the current instance of the level
  local level = getLevel()
```

```
-- Add our Monocle entity to the level's list (inherited from Scene, which has
Level:add(coroutineEntity)
end
```

Now, let's make some coroutines! Looking at `Monocle.Entity`, we can see it has a very similar `Add()` method, but this time accepting a `Component` as its parameter instead of a `Monocle.Entity`.

现在，让我们创建一些协程！查看 `Monocle.Entity`，我们可以看到它有一个非常相似的 `Add()` 方法，但这次接受一个 `Component` 作为参数而不是 `Monocle.Entity`。

```
/// <summary>
/// Shortcut function for adding a Component to the Entity's Components list.
/// </summary>
/// <param name="component">The Component to add.</param>
public void Add(Component component)
{
    // ...
}
```

And what do you know, if we look at `Monocle.Coroutine` — the very base level of coroutine code — we can see that it extends `Component`.

我们来看看 `Monocle.Coroutine` ——协程代码的最基础层级——可以发现它扩展了 `Component`。

```
public class Coroutine : Component
{ ... }
```

If a class extends another in C#, it's kind of considered to be both its own class and its "superclass" (the one it extends) at once. `Monocle.Entity.Add()` accepts a `Component` — `Monocle.Coroutine` isn't a `Component`, but it extends `Component`, so it can still be passed to this method. It's both a `Component` and a `Monocle.Coroutine` simultaneously.

在 C# 中，如果一个类扩展了另一个类，它既被视为自己的类，也同时被视为其“超类”（即它所扩展的类）。`Monocle.Entity.Add()` 接受一个 `Component` —— `Monocle.Coroutine` 不是 `Component`，但它扩展了 `Component`，所以它仍然可以被传递给这个方法。它同时既是 `Component` 也是 `Monocle.Coroutine`。

Let's look at one specific method in `Monocle.Coroutine`. This method is called the “constructor,” and it's called whenever we create a new instance of `Monocle.Coroutine`.

让我们看看 `Monocle.Coroutine` 中的一个特定方法。这个方法被称为“构造函数”，每当创建 `Monocle.Coroutine` 的新实例时就会调用它。

```
public Coroutine(IEnumerator functionCall, bool removeOnComplete = true)
{...}
```

With this, everything's fallen into place. When we create a `Monocle.Coroutine`, we pass it a function (the `IEnumerator functionCall` parameter). This function needs to return a value of type `IEnumerator` — this is why the coroutine functions we've seen before all return that type. With that function passed to the constructor, we've successfully created a coroutine.

有了这些，一切都清晰了。当我们创建一个 `Monocle.Coroutine` 时，我们会传递给它一个函数（即 `IEnumerator functionCall` 参数）。这个函数需要返回一个类型为 `IEnumerator` 的值——这就是为什么我们之前看到的协程函数都返回那种类型。当这个函数被传递给构造函数时，我们就成功创建了一个协程。

Finally, let's break down the last bit of Lua code, going from the inside out:
最后，让我们从内到外逐步解析这最后一段 Lua 代码：

```
local monocle = require("#monocle")
local coroutineEntity = monocle.Entity()

function onBegin()
    local level = getLevel()
    -- Add our Monocle entity to the level's list
    level:add(coroutineEntity)
    coroutineEntity:add(monocle.Coroutine(wait(4)))
    coroutineEntity:add(monocle.Coroutine(say("DIALOGUE")))
end
```

Step 1. `wait()` is a function that returns an `IEnumerator` value.

步骤 1. `wait()` 是一个返回 `IEnumerator` 值的函数。

Step 2. When we pass it to `monocle.Coroutine`, the constructor is run. It takes that `wait()` function and makes it a coroutine.

步骤 2. 当我们将其传递给 `monocle.Coroutine` 时，构造函数会被执行。它将这个 `wait()` 函数转换为一个协程。

Step 3. Our `coroutineEntity` adds the newly created `monocle.Coroutine` and its own newly created `wait()` coroutine to its list of `Components`.

步骤 3. 我们的 `coroutineEntity` 将新创建的 `monocle.Coroutine` 和它自己新创建的 `wait()` 协程添加到它的 `Component` 列表中。

Step 4. The `coroutineEntity` has been added to the `Level`'s list of coroutine creators. `Level` sees that `coroutineEntity` has just added a new `Component` to

its list.

第 4 步。 `coroutineEntity` 已被添加到 `Level` 的协程创建者列表中。 `Level` 看到 `coroutineEntity` 刚刚向其列表中添加了一个新的 `Component` 。

Step 5. Steps 1–4 repeat with the `say()` function.

第 5 步。步骤 1 至 4 将重复执行 `say()` 函数。

Step 6. `Level` runs both of the functions included on `coroutineEntity`'s list simultaneously.

第 6 步。 `Level` 同时运行 `coroutineEntity` 列表中包含的两个函数。

Step 7. Your functions are now running in parallel! Congratulations!

第 7 步。您的函数现在正在并行运行！恭喜！

To conclude this section, let's go all the way back to our initial example of zooming and moving the camera simultaneously.

要总结这一部分，让我们回到最初同时进行缩放和移动摄像头的例子。

```
local monocle = require("#monocle")
local coroutineEntity = monocle.Entity()

function onBegin()
    local level = getLevel()
    level:add(coroutineEntity)
    coroutineEntity:add(monocle.Coroutine(celeste.CutsceneEntity.CameraTo(vector2(
    coroutineEntity:add(monocle.Coroutine(level.ZoomTo(vector2(160, 190), 1.5, 5))
end
```

Since `CameraTo` and `ZoomTo` both return `IEnumerator` values, we can put them inside the “parallel cutscene wrapper” no problem. This will move and zoom the camera at the exact same time. It was a long journey, but we got there.

由于 `CameraTo` 和 `ZoomTo` 都返回 `IEnumerator` 值，我们可以没有问题地将它们放入“并行过场动画包装器”中。这将同时移动和缩放摄像头。这是一段漫长的旅程，但我们最终做到了。

Section TL;DR 本节要点

To run functions in parallel rather than in sequence, we need to recreate `coroutine.yield` ourselves. This results in a lengthier but equally functional “parallel coroutine wrapper,” shown below. Only functions that are coroutines (those that return `IEnumerator`) can go in this wrapper. The rest of this section was just explaining how it works on a technical level.

为了并行运行函数而不是按顺序运行，我们需要自己重新创建 `coroutine.yield`。这会导致一个更长但功能相同的“并行协程包装器”，如下所示。只有协程函数（那些返回 `IEnumerator` 的函数）才能放入这个包装器中。本节其余部分只是从技术层面解释了它是如何工作的。

```

-- Sequential coroutine wrapper
function onBegin()
    coroutine.yield(...)
end

-- Parallel coroutine wrapper
local monocle = require("#monocle")
local coroutineEntity = monocle.Entity()

function onBegin()
    local level = getLevel()
    level:add(coroutineEntity)
    -- These next 2 lines will run in parallel
    coroutineEntity:add(monocle.Coroutine(...))
    coroutineEntity:add(monocle.Coroutine(...))
end

```

Chapter 6: Fully Functional

第六章：完全功能

Let's take a step down in complexity for a bit. So far, we've only used existing functions — `onBegin()`, `onEnd()`, `ZoomTo()`, `Play()`, `say()`, and more. Why don't we make our own?

让我们稍微降低一下复杂度。到目前为止，我们只使用了现有的函数——

`onBegin()`、`onEnd()`、`ZoomTo()`、`Play()`、`say()`，等等。我们为什么不自己创建一些呢？

Making our own Lua functions is surprisingly simple:

自己创建 Lua 函数出乎意料地简单：

```

local function xyz()
    ...
end

function onBegin()
    ...
end

```

Since we're defining the functions ourselves, we want to prefix them with `local`, just like we do for variables. The only other specification is that they must come *before* `onBegin()` in the file — otherwise, you can do pretty much anything you want with them. We can even give them parameters.

既然是我们自己定义函数，我们需要像变量一样给它们加上 `local` 前缀。唯一的其他要求是它们必须在文件中的 `onBegin()` 之前——否则，你可以几乎用它们做任何你想做的事情。我们甚至可以给它们传递参数。

```

-- This function will say the specified dialogue ID, then wait 5 seconds.
local function talk(id)
    say(id)

```

```
wait(5)
end
```

Let's say we have a cutscene where we want the player to dash right 5 times. The simplest possible way to make the player dash in Lua Cutscenes takes 5 lines of code. We could copypaste that 5 times, or we could just make dashing a function and call that 5 times instead.

假设我们有一个过场动画，需要让玩家向右冲刺 5 次。在 Lua Cutscenes 中让玩家冲刺的最简单方法需要 5 行代码。我们可以复制粘贴 5 次，或者我们可以把冲刺做成一个函数，然后调用 5 次。

```
local function dash(xVector, yVector)
    player.OverrideDashDirection = vector2(xVector, yVector)
    setPlayerState(2)
    wait(0.6)
    setPlayerState(11)
    player.OverrideDashDirection = nil
end

function onBegin()
    dash(1,0)
    dash(1,0)
    dash(1,0)
    dash(1,0)
    dash(1,0)
end
```

This is still kind of annoying, though —remember all the way back at the beginning when we talked about loops? Now's the time to use them.

这仍然有点烦人——还记得我们一开始讨论循环的时候吗？现在就是使用它们的时候了。

```
local function dash(xVector, yVector)
    player.OverrideDashDirection = vector2(xVector, yVector)
    helpers.setPlayerState (2)
    helpers.wait(0.6)
    helpers.setPlayerState (11)
    player.OverrideDashDirection = nil
end

function onBegin()
    for i=0,5,1 do
        dash(1,0)
    end
end
```

Like loops, functions allow you to simplify your cutscene and reduce overall redundancy. However, functions have one additional property which is why they're being talked about all the way at the end here — you can actually

make them into coroutines.

和循环一样，函数可以简化你的过场动画并减少整体冗余。然而，函数有一个额外的特性，这也是为什么它们被放在这里讨论的原因——你可以将它们变成协程。

Let's say we want to have a cutscene where the camera pans and zooms as dialogue plays. If the player mashes through the dialogue, they could potentially finish it before the zoom ends, and the cutscene would continue as normal instead of waiting for the zoom to be done. Here's where making our own coroutine comes in — we can make one that serves as a timer!

假设我们想要一个过场动画，其中摄像机在对话播放时平移和缩放。如果玩家快速点击对话，他们可能会在缩放结束之前完成对话，而过场动画会像正常一样继续，而不是等待缩放完成。这就是我们创建自己的协程的地方——我们可以创建一个作为计时器！

```
-- Get some built-in lua cutscenes help (we'll need it)
local lua_helper = celeste.Mod.LuaCutscenes.LuaHelper

-- Set up parallel execution
local monocle = require("#monocle")
local coroutineEntity = monocle.Entity()

-- Create a variable to store our timer coroutine
local waitingCoroutine
-- Create a variable that will become true when the timer is over
local goodToGo = false
```

We need to get the Lua Helper in order to make our own Lua code into coroutines. Next, we set up all the requirements for parallel execution, then we create a variable to “hold” the coroutine. This should seem off, because usually it's methods that have code inside them, not variables or fields. This is a great point that I don't have an answer to. I know this works; I do not know how. Don't worry about it.

我们需要使用 **Lua Helper** 将我们的 **Lua** 代码变成协程。接下来，我们设置所有并行执行的要求，然后我们创建一个变量来“持有”协程。这看起来有点奇怪，因为通常方法里面才有代码，而不是变量或字段。这是一个很好的问题，但我没有答案。我知道这能工作，但不知道原理。别担心。

Once we've got everything set up, we create our timer function:

一旦我们设置好所有内容，我们就创建我们的计时器函数：

```
local function waitTilCameraMoved()
    wait(5)
    goodToGo = true
end
```

Simple enough — all we do is wait and then set a variable to true. Notably, setting a variable to true is not something that takes time (and thus not a coroutine on its own), but we're going to run it in parallel anyway because we're going to make this entire thing a coroutine.

很简单——我们只是等待，然后将一个变量设置为 `true`。值得注意的是，将变量设置为 `true` 并不是一个耗时操作（因此它本身不是一个协程），但我们仍然会并行运行它，因为我们要将整个事情作为一个协程来实现。

This next function allows us to do just that:

下一个函数允许我们做到这一点：

```
-- Code taken from microlith (thanks!)
local function makeCoroutine(func)
    return monocle.Coroutine(lua_helper.LuaCoroutineToIEnumerator(coroutine.create
end
```

Lua Helper does some behind the scenes magic and will make any function passed into *this* function into something that returns an `IEnumerator`. This will allow us to run it inside the parallel cutscene wrapper.

Lua Helper 在幕后进行一些魔法操作，并将传递给这个函数的任何函数变成返回 `IEnumerator` 的函数。这将允许我们在并行场景包装器内部运行它。

Now we move to `onBegin()`.

现在我们进入 `onBegin()`。

```
function onBegin()
    getLevel():add(coroutineEntity)
    coroutineEntity:add(monocle.Coroutine(celeste.CutsceneEntity.CameraTo(vector2(
    coroutineEntity:add(monocle.Coroutine(getLevel():ZoomTo(vector2(120, 120), 1.5
    waitingCoroutine = makeCoroutine(waitTilCameraMoved)
    cutsceneEntity:Add(waitingCoroutine)
    say("DIALOGUE_1")
    while not goodToGo do
        wait()
    end
    say("DIALOGUE_2")
end
```

The first 3 lines should be familiar at this point — they're just moving and zooming the camera in parallel. It's the lines after that which are new. We give our `waitingCoroutine` variable that we created earlier the newly created coroutine to hold by putting the name of the function into `makeCoroutine()`. `CutsceneEntity` conveniently extends `Monocle.Entity`, giving it all the properties we talked about in the above section; this is why we can call its

`Add()` method and add the variable holding the new coroutine to its list.

前 3 行此时应该很熟悉了——它们只是并行移动和缩放相机。新内容是在那之后的行。我们将之前创建的 `waitingCoroutine` 变量赋予新创建的协程来持有，通过将函数的名称放入 `makeCoroutine()` 中。`CutsceneEntity` 方便地扩展了 `Monocle.Entity`，赋予它我们在上一节讨论的所有属性；这就是为什么我们可以调用它的 `Add()` 方法，并将持有新协程的变量添加到它的列表中。

Finally, we throw in a loop in order to make sure that we don't continue the cutscene until the timer has expired. Once `waitTilCameraMoved()` has finished running, that `goodToGo` variable will be true, and the next line of dialogue can play.

最后，我们加入一个循环，以确保在计时器到期之前不会继续播放片段。一旦 `waitTilCameraMoved()` 完成运行，那个 `goodToGo` 变量就会变为真，然后可以播放下一行对话。

Once the cutscene is over, we need to make sure we're not causing any memory leaks by leaving any unnecessary elements on the list.

片段结束后，我们需要确保不会因在列表中留下任何不必要的元素而导致内存泄漏。

```
function onEnd()
    -- check if the coroutine even exists - the player could have skipped the
    -- cutscene before it was created
    if waitingCoroutine then
        -- stop the coroutine
        waitingCoroutine:Cancel()
        -- remove it from the list of coroutine creators
        cutsceneEntity:Remove(waitingCoroutine)
    end
end
```

All together: 全部一起:

```
local lua_helper = celeste.Mod.LuaCutscenes.LuaHelper

local monocle = require("#monocle")
local coroutineEntity = monocle.Entity()

local waitingCoroutine
local goodToGo = false

local function waitTilCameraMoved()
    wait(5)
    goodToGo = true
end

local function makeCoroutine(func)
    return monocle.Coroutine(lua_helper.LuaCoroutineToIEnumerator(coroutine.create
end

function onBegin()
    getLevel():add(coroutineEntity)
    coroutineEntity:add(monocle.Coroutine(celeste.CutsceneEntity.CameraTo(vector2(
    coroutineEntity:add(monocle.Coroutine(getLevel():ZoomTo(vector2(120, 120), 1.5
    waitingCoroutine = makeCoroutine(waitTilCameraMoved)
```

```

cutsSceneEntity:Add(waitingCoroutine)
say("DIALOGUE_1")
while not goodToGo do
    wait()
end
say("DIALOGUE_2")
end

function onEnd()
    if waitingCoroutine then
        waitingCoroutine:Cancel()
        cutsSceneEntity:Remove(waitingCoroutine)
    end
end
end

```

This example shows how you can run elements both in parallel and in sequence simultaneously. We need a bunch of stuff to run at once, but we also need to wait for it to all be done to move forward. If you master parallel and sequential execution like this, you'll be making fantastic cutscenes in no time.

这个例子展示了如何在并行和顺序的同时运行元素。我们需要同时运行很多东西，但我们还需要等待所有事情完成才能继续前进。如果你掌握了并行和顺序执行，你很快就能制作出精彩的过场动画。

Section TL;DR TL;DR 部分

We can make our own functions to reduce redundancy in our code; they need to start with the `local` keyword and come before `onBegin()` and `onEnd()` in the file. We can make these functions into coroutines using Lua Helper, which allows us to use non- `IEnumerator` returning functions in parallel execution setups. One example shown in this section is a simple timer. By creating our own coroutines from Lua code, we can accomplish simultaneous parallel and sequential execution.

我们可以通过创建自己的函数来减少代码中的冗余；它们需要以 `local` 关键字开头，并在文件中的 `onBegin()` 和 `onEnd()` 之前。我们可以使用 **Lua Helper** 将这些函数转换为协程，这允许我们在并行执行设置中使用非 `IEnumerator` 返回函数。本节中展示的一个例子是一个简单的计时器。通过从 Lua 代码创建自己的协程，我们可以实现同时的并行和顺序执行。

Chapter 7: Concluding Statements and Examples

第七章：结论性陈述和示例

By now I've written over 9000 words on this topic, which was 8500 more than I was originally planning to write. Oops!

到目前为止，我已经在这个主题上写下了超过 9000 字，这比最初计划的 8500 字多了很多。哎呀！

Hopefully this has been generally helpful for all your Lua needs. There's no shame in looking at existing cutscenes for help — they're usually packaged in the Assets folder of released mods — and please do ask in #modding-help in Celestecord if you need any help. I made fun of them a while earlier but they

are genuinely helpful.

希望这能对你所有的 Lua 需求有所帮助。参考现有的过场动画并不丢人——它们通常打包在已发布的模组的 Assets 文件夹中——如果你需要任何帮助，请务必在 Celestecord 的 #modding-help 频道中提问。我之前曾嘲笑过他们，但他们确实非常乐于助人。

Special thanks to Luna for writing the bulk of the first C# section after my initial draft was kind of miserable, JaThePlayer for doing the same for the second C# section on parallel execution, and microlith for its Lua Cutscenes Recipe Book. It's more focused on examples than this, making it really a great resource. [Go read it for more!](#)

特别感谢 Luna 在我初稿相当糟糕后，撰写了大部分第一部分 C# 内容；感谢 JaThePlayer 为我撰写了关于并行执行的第二部分 C# 内容；感谢 microlith 提供的 Lua Cutscenes Recipe Book。这本书更侧重于示例，使其成为一个非常棒的资源。去读它获取更多信息吧！

I'm going to cover a few more miscellaneous cases now that I wish somebody had written documentation for when I was starting out. I won't be explaining these in a lot of detail though, it'll mostly just be code blocks. Good luck and have fun lua cutsceneing! I am not responsible for onEnd coroutine crashes.

我现在将涵盖一些我希望在刚开始时有文档说明的杂项案例。不过，我不会详细解释这些内容，大部分只是代码块。祝你好运，享受 Lua 过场动画制作吧！我不对 onEnd 协程崩溃负责。

Making a cutscene only run once

让过场动画只播放一次

```
function onBegin()
    if not (getFlag("gate_flag")) then
        ...
        setFlag("gate_flag", true)
    end
end

function onEnd()
    setFlag("gate_flag", true)
end
```

Change player facing direction

改变玩家朝向

```
-- Facing directions are enums, which is why I didn't include them in
-- this tutorial. Too complicated to cover easily. To use enums in Lua (which
-- doesn't normally support them), use the getEnum method built into Lua
-- Cutscenes. Thanks Cruroror.
function onBegin()
    -- Face left
    player.Facing = getEnum("Celeste.Facings", "Left")
    -- Wait 1 second
```

```

wait(1)
-- Face right
player.Facing = getEnum("Celeste.Facings", "Right")
end

```

Making dialogue choices with the ChoiceDialog function

使用 ChoiceDialog 函数制作对话选择

```

-- Code by Ja
-- It accepts a table containing information about all possible choices
-- and their requirements.
choiceDialog({
  -- The most basic dialog choice.
  -- The dialog key DIALOG_1 will be used for displaying the choice,
  -- then when you choose it, the dialog DIALOG_1_SAY will be shown.
  { "DIALOG_1" },
  {
    -- This dialog requires that you had already chosen "DIALOG_1" before
    "DIALOG_1_PART_2",
    requires = { "DIALOG_1" }
  },
  {
    -- This dialog will always be available, no matter how many times it
    "REPEATING",
    repeatable = true
  },
  {
    -- 'requires' can also be a function returning a bool. If this funct
    "RANDOM",
    requires = function ()
      return math.random() > 0.5
    end,
  },
  {
    -- There can be multiple requirements for the same choice.
    -- All conditions need to be met at the same for this choice to show
    "AFTER_1_AND_RANDOM",
    requires = { "DIALOG_1", "RANDOM", function ()
      return math.random() > 0.5
    end},
  },
  {
    -- This choice will not display a dialog, and instead run the code i
    "DIALOG_WITH_EFFECT",
    onChosen = function ()
      jump(0.2)
      wait()
      waitUntilOnGround()
    end
  },
  {
    --[[
    This choice will be available after either DIALOG_1 or DIALOG_WITH_E
    ctx is a table containing information about the current choice dialo
    ctx.usedDialogs is a table which stores which dialogs you already us
    ["DIALOG_KEY"] = true,
    ...
    ]]]
    "AFTER_1_OR_EFFECT",
    requires = function (ctx)
      return ctx.usedDialogs["DIALOG_1"] or ctx.usedDialogs["DIALOG_WI
    end
  },
  {
    -- this choice will still display a dialog like normal, but also run
    "DIALOG_CLOSE",
    onEnd = function ()
      -- call this function to close the choice dialog and allow the c
      closeChoiceDialog()
    end
  }
}

```

```
}  
})
```

Adding an NPC 添加 NPC

```
-- Code from microlith and bit  
local function summonTheo(position)  
    theo = celeste.NPC(position)  
    getRoom():Add(theo)  
    local sprite = celeste.GFX.SpriteBank:Create("theo")  
    theo:Add(sprite)  
    theo.Sprite = sprite  
    sprite:Play("idle")  
    theo.IdleAnim = "idle"  
    theo.MoveAnim = "walk"  
    theo.Maxspeed = 58  
    theo.MoveY = true  
end  
  
local function moveTheo(xVector,yVector)  
    coroutine.yield(theo:MoveTo(theo.Position + vector2(xVector, yVector)))  
end
```

Less complicated dash function

更简单的闪避功能

```
-- Code from mosscairn  
local vector2 = require("#microsoft.xna.framework.vector2")  
function dash(xVector,yVector)  
    player.OverrideDashDirection = vector2(xVector, yVector)  
    helpers.setPlayerState (2)  
    helpers.wait(0.6)  
    helpers.setPlayerState (11)  
    player.OverrideDashDirection = nil  
end
```

i am sumping. like if you agree

我在跳跃。比如如果你同意

Lua

Celeste

Cut Scenes



Written by Gamati 作者: Gamati

7 followers · 1 following

Follow

Responses (2) 评论 (2)





glll

What are your thoughts?



Gamation Author

Jul 30, 2023

i am the author and i think i did a good job :)

我是作者，我认为我做得很好 :)



1

[Reply](#)



Saplonily Mason

Oct 1, 2023 (edited)

Hi, I'm reading this tourial and thanks for writing this. But with finding out some problems, for example:

嗨，我正在阅读这个教程，谢谢你的写作。但发现一些问题，例如：

...

...

```
coroutine.yield(level.ZoomTo(vector2(150, 150), 2, 3.5)
```

```
-- While maintaining this zoom level, focus on the point (300, 300) over the
```

```
-- 在保持这个缩放级别的同时，将焦点放在点 (300, 300) 上
```

```
-- next 3 seconds -- 接下来 3 秒
```

```
coroutine.yield(celeste.CutsceneEntity.CameraTo(vector2(300, 300), 3)
```

...

...

This section of code actually has some grammarly issues, such as there's no pairing closing parenthesis of "coroutine.yield(". Hope to be fixed, thanks! :)

这段代码实际上存在一些语法问题，比如"coroutine.yield("没有配对的关闭括号。希望得到修复，谢谢！ :)



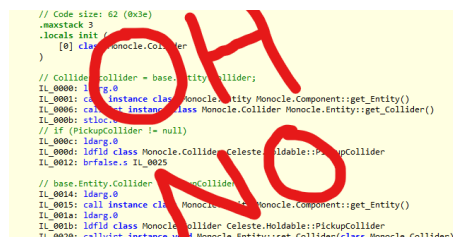
[Reply](#)

More from Gamation 更多来自 Gamation



Gamation

Celeste Camera in 3 Levels of Complexity



Gamation